Universidad / Institución: Castro Carazo

Curso : Desarrollo Web con Angular / Programación 5

Título del Proyecto: Asignación 1

Estudiante: Sebastián Alberto Naranjo Salazar

Fecha: Octubre 2026

2. Desarrollo

Descripción del Proyecto

El objetivo del proyecto fue desarrollar una aplicación web reactiva en Angular utilizando componentes independientes (*Standalone Components*), control de flujo moderno (@for, @if), el concepto de Signals para el manejo del estado local y consumo de datos asíncronos mediante HttpClient.

Arquitectura e Implementación

La aplicación se estructuró siguiendo las buenas prácticas de Angular separando responsabilidades:

1. **Modelos (models/character.ts):**

Se definieron las interfaces TypeScript para tipar la respuesta de la API (CharacterResponse) y la estructura de cada personaje (Character), garantizando la seguridad de tipos en la aplicación.

2. **Servicio HTTP (services/dragonball.ts):**

Se implementó DragonBallService utilizando la función inject(HttpClient) para realizar peticiones GET a la API pública de Dragon Ball (https://dragonball-api.com/api/characters).

3. **Lógica del Componente (components/character-list/character-list.ts):**

   - Se manejaron dos *Signals*: characters = signal<Character[]>([]) para almacenar la lista y loading = signal<boolean>(true) para indicar el estado de carga.
   - Se realizó la suscripción dentro de ngOnInit() para poblar los personajes al inicializar la vista.

4. **Plantilla HTML e Interacción (components/character-list/character-list.html):**

   - Se empleó la sintaxis @for para iterar sobre la señal de personajes (characters()).
   - Se utilizó un contenedor con dos vistas (first-content y second-content) basadas en un diseño adaptado de Uiverse.io.
   - **Comportamiento:** Mientras el cursor está fuera, muestra la imagen y el nombre del personaje; al pasar el cursor sobre

la tarjeta (:hover), se aplica una animación de rotación y despliega los detalles de combate (Raza, Ki).

5. **Hoja de Estilos CSS (components/character-list/character-list.css):**

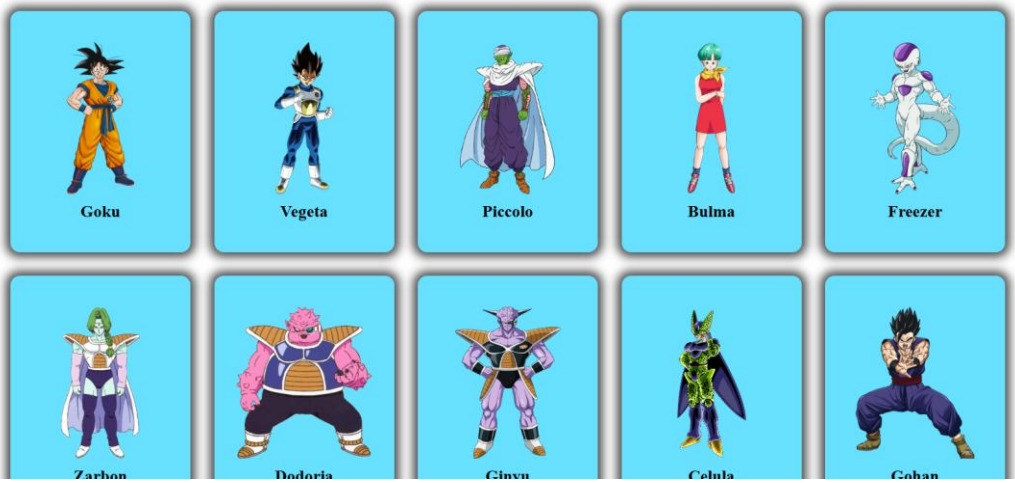
Se maquetó una cuadrícula responsiva con flex-wrap y animaciones CSS de transición (transition, transform: rotate, scale).

6. **Integración Principal (app.ts y app.html):**

Se importó e integró <app-character-list></app-character-list> dentro de la raíz de la aplicación para mostrar la lista como pantalla principal.

Evidencia

Personajes de Dragon Ball




```typescript
import { Component, OnInit, inject, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { DragonBallService } from '../../services/dragonball';
import { Character } from '../../models/character';

@Component({
  selector: 'app-character-list',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './character-list.html',
  styleUrl: './character-list.css'
})
export class CharacterListComponent implements OnInit {
  private dbService = inject(DragonBallService);

  characters = signal<Character[]>([]);
  loading = signal<boolean>(true);

  ngOnInit(): void {
    this.dbService.getCharacters(1, 12).subscribe({
      next: (data) => {
        this.characters.set(data.items);
        this.loading.set(false);
      },
      error: (err) => {
        console.error('Error al cargar personajes:', err);
        this.loading.set(false);
      }
    });
```

5. Referencias

1. **Dragon Ball API Documentation:**

Recuperado de: https://dragonball-api.com/

2. **Angular Documentation (Signals & Control Flow):**

Recuperado de: https://angular.dev/

3. **Uiverse.io Card Component Design (suleymanlaarabidev):**

Diseño e inspiración para tarjetas interactivas CSS. Recuperado de: https://uiverse.io/